

LISTAS ENCADEADAS (Prática)

Depuração de uma Lista Encadeada com Erros Lógicos.

Descrição:

O código fornecido implementa todas as operações básicas de uma lista encadeada simples em C. No entanto, ele contém **três erros lógicos sutis** que não impedem a compilação, mas causam **comportamentos incorretos na execução**.

Tarefa:

1. Observe e descreva o comportamento incorreto com base nas chamadas no main e saídas produzidas (veja no final deste exercício).
2. Localize e corrija os bugs no código.
3. Explique, em poucas linhas, **por que cada bug causava o comportamento observado**.

Código:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int value;
    struct Node* next;
} Node;

Node* create_node(int value) {
    Node* new_node = (Node*) malloc(sizeof(Node));
    new_node->value = value;
    new_node->next = NULL;
    return new_node;
}
```

```
Node* insert_front(Node* head, int value) {  
    Node* new_node = create_node(value);  
    new_node->next = NULL; // BUG: deveria ser 'new_node->next = head;'  
    return new_node;  
}
```

```
Node* insert_back(Node* head, int value) {  
    Node* new_node = create_node(value);  
    if (head == NULL) return new_node;  
  
    Node* current = head;  
    while (current->next != NULL)  
        current = current->next;  
  
    current->next = new_node;  
    return head;  
}
```

```
Node* insert_at(Node* head, int value, int pos) {  
    if (pos <= 0 || head == NULL)  
        return insert_front(head, value);  
  
    Node* current = head;  
    int index = 0;  
  
    while (current->next != NULL && index != pos - 1) {  
        current = current->next;  
        index++;  
    }  
}
```

```
Node* new_node = create_node(value);
new_node->next = current->next;
current->next = new_node;
return head;
}
```

```
Node* remove_front(Node* head) {
    if (head == NULL) return NULL;
    Node* temp = head;
    head = head->next;
    free(temp);
    return head;
}
```

```
Node* remove_back(Node* head) {
    if (head == NULL) return NULL;

    if (head->next == NULL) {
        free(head);
        head = NULL;
        // BUG: faltou 'return head;' aqui
    }
}
```

```
Node* current = head;
while (current->next->next != NULL)
    current = current->next;

free(current->next);
current->next = NULL;
return head;
}
```

```

Node* remove_at(Node* head, int pos) {
    if (head == NULL || pos < 0) return head;
    if (pos == 0) return remove_front(head);

    Node* current = head;
    int index = 0;
    while (current->next != NULL && index < pos - 1) {
        current = current->next;
        index++;
    }

    if (current->next == NULL) return remove_back(head);
    Node* temp = current->next;
    current->next = temp->next;
    free(temp);
    return head;
}

```

```

Node* search(Node* head, int value) {
    Node* current = head;
    while (current != NULL) {
        if (current->value == value)
            return current;
        current = current->next;
    }
    return NULL;
}

```

```
void free_list(Node* head) {  
    free(head);  
}
```

```
void print_list(Node* head) {  
    Node* current = head;  
    printf("List: ");  
    while (current != NULL) {  
        printf("%d -> ", current->value);  
        current = current->next;  
    }  
    printf("NULL\n");  
}
```

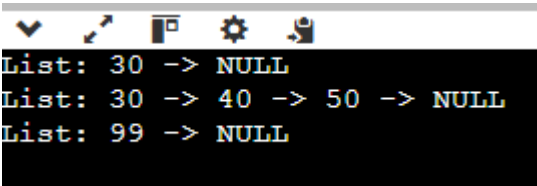
```
int main() {  
    Node* head = NULL;  
  
    head = insert_front(head, 10);  
    head = insert_front(head, 20);  
    head = insert_front(head, 30);  
    print_list(head);  
  
    head = insert_back(head, 40);  
    head = insert_back(head, 50);  
    print_list(head);  
  
    free_list(head);  
    head = NULL;  
  
    head = insert_back(head, 99);  
    print_list(head);  
  
    head = remove_back(head);
```

```
print_list(head);

free_list(head);

return 0;
}
```

Saída:

A terminal window with a dark background and light-colored text. The window has a title bar with standard icons (minimize, maximize, close, settings, and a user icon). The output shows three lines of text: 'List: 30 -> NULL', 'List: 30 -> 40 -> 50 -> NULL', and 'List: 99 -> NULL'.

```
List: 30 -> NULL
List: 30 -> 40 -> 50 -> NULL
List: 99 -> NULL
```